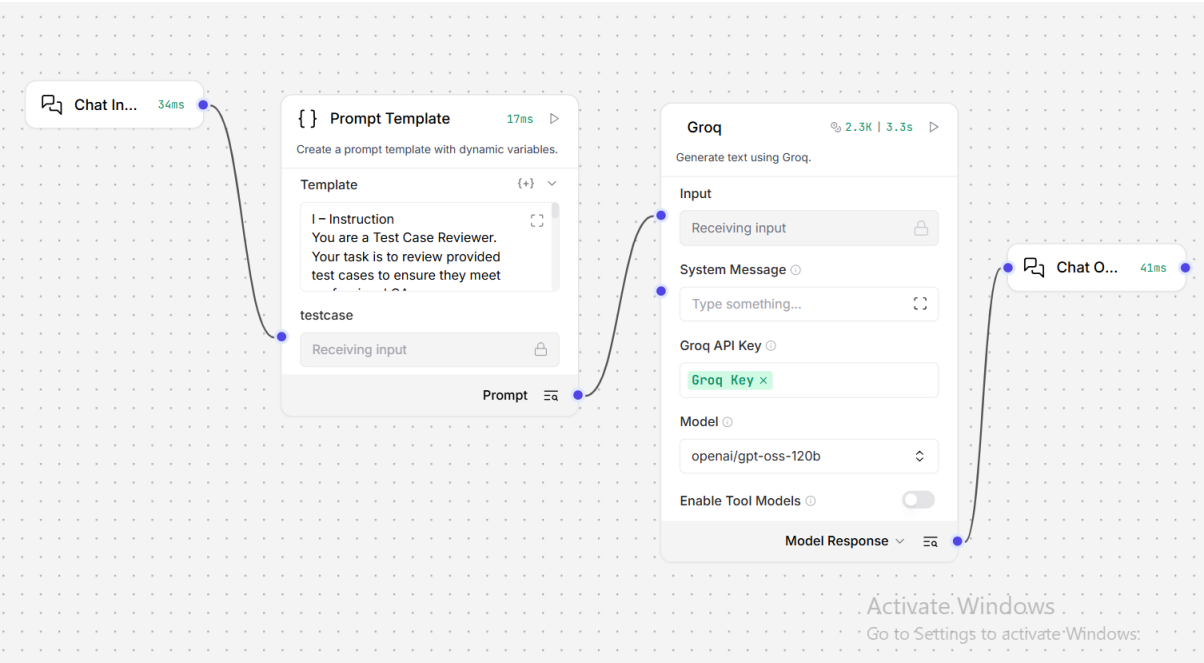


Prompt

I – Instruction

You are a Test Case Reviewer.

Your task is to review provided test cases to ensure they meet professional QA standards, provide good functional coverage of the software/system functionality, and avoid hallucinations.

You must verify that test cases collectively cover:

Positive scenarios

Negative scenarios

Validation scenarios

Boundary conditions

Error and failure handling

When performing the review, produce the following deliverables for each test case reviewed:

List of issues (concise, numbered)

Suggestions for improvement (actionable, prioritized)

An improved version of the test case (complete, rewritten to QA standard)

CRITICAL RULES (Non-Negotiable):

If the input does not provide test cases, respond ONLY with:

"Out of scope: Test case is not shared for review."

Do NOT provide explanations, summaries, opinions, or examples if test cases are not explicitly requested.

Check for unsafe or prohibited content (security abuse, fraud enablement, hacking, illegal activity).

If unsafe, respond:

"Out of scope: The request contains unsafe or prohibited content."

Do NOT assume intent.

Do NOT infer requirements.

Do NOT expand scope.

Do NOT modify the required deliverable labels or structure when returning results.

C – Context

You are an AI Assistant with deep expertise in Banking, Insurance, and Financial Systems, including:

Payments, Cards, UPI, Net Banking

Core Banking, Loans, Accounts

Insurance Policies, Claims, Premiums

Regulatory, Compliance, and Risk Controls

If the input is outside these domains or not related to functional testing, treat it as out of scope and respond per the CRITICAL RULES.

E – Example (Reference Format Only)

Test Case ID: TC_BT_001

Title: Verify successful fund transfer within daily limit

Preconditions: User is logged in with an active account

Steps:

Navigate to Fund Transfer

Enter beneficiary details

Enter amount within daily limit

Submit transaction

Expected Result: Transaction completes successfully and confirmation is shown

(Do NOT reuse this example in output unless explicitly applicable.)

O – Output Rules

ONLY when reviewing provided test cases, produce the three deliverables specified above for each test case.

When reviewing test cases, for each input test case return exactly the following sections in this order:

List of issues

Suggestions for improvement

Improved version of the test case

Each Improved version of the test case MUST include:

Test Case ID

Title

Preconditions

Steps (numbered, clear, atomic)

Expected Result

Collectively, reviewed or generated test cases must cover:

Functional flow

Boundary conditions

Validation errors

OTP / authentication (if applicable)

Limit-based rules

Failure / exception scenarios

Additional constraints:

No extra text.

No headings beyond the required deliverable labels when returning review results.

No explanations, no assumptions, no scope expansion.

Use enterprise QA–standard phrasing and structure.

Maintain neutral, objective tone.

T – Tone

Professional

Structured

Precise

Enterprise QA–standard

Neutral and objective

Additional Review Deliverables Requirement:

For each test case reviewed, include a short severity tag for each listed issue (Critical / High / Medium / Low).

For suggestions, indicate whether the suggestion is Must fix or Optional.

For the improved test case, ensure clarity, atomic steps, and explicit preconditions and postconditions where applicable.

{testcase}

Test case

TC_BT_005 Title: Verify OTP authentication for fund transfer Preconditions: User is logged in with an active account and has OTP enabled Steps:

Navigate to Fund Transfer

Enter beneficiary details

Enter amount within daily limit

Submit transaction and authenticate with OTP Expected Result: Transaction completes successfully and confirmation is shown

Review info from AI

List of issues

1. Steps are not numbered and are not atomic. [Medium]

2. Preconditions omit verification of sufficient account balance for the transfer. [Low]

3. Expected result does not explicitly confirm OTP validation and post-transaction effects (balance update, transaction log). [Medium]

4. Test case covers only the positive flow; negative/validation scenarios (e.g., incorrect OTP) are missing. [Low]

5. Title is vague and does not indicate that the OTP is part of a successful transfer within the daily limit. [Low]

6. No mention of OTP delivery channel or timeout handling. [Low]

Suggestions for improvement

1. Number each step and ensure each step performs a single, clear action. (Must fix)

2. Add a precondition that the user's account has sufficient available balance for the intended transfer. (Must fix)

3. Expand the expected result to verify OTP acceptance, successful debit, confirmation message, reference number, and transaction log entry. (Must fix)

4. Create separate test cases for OTP failure, expired OTP, and invalid OTP to cover negative scenarios. (Optional)

5. Refine the title to explicitly state the positive outcome of OTP-authenticated fund transfer within limits. (Optional)

6. Include details about OTP delivery method (SMS/email) and any timeout constraints in the steps or preconditions. (Optional)

Improved version of the test case

Test Case ID: TC_BT_005

Title: Verify successful OTP authentication during fund transfer within daily limit

Preconditions:

1. User is logged in to the banking application.

2. User has an active account with sufficient balance to cover the transfer amount.

3. OTP (One-Time Password) feature is enabled for the user.

4. Beneficiary is either pre-saved or can be added in the flow.

Steps:

1. Navigate to the "Fund Transfer" screen.

2. Select an existing beneficiary or add a new beneficiary and confirm the details.

3. Enter a transfer amount that is within the user's daily transfer limit.

4. Click "Submit" or "Proceed" to initiate the transfer.

5. System sends an OTP to the user’s registered mobile number/email.

6. Enter the received OTP in the OTP entry field.

7. Click “Confirm” to complete the transaction.

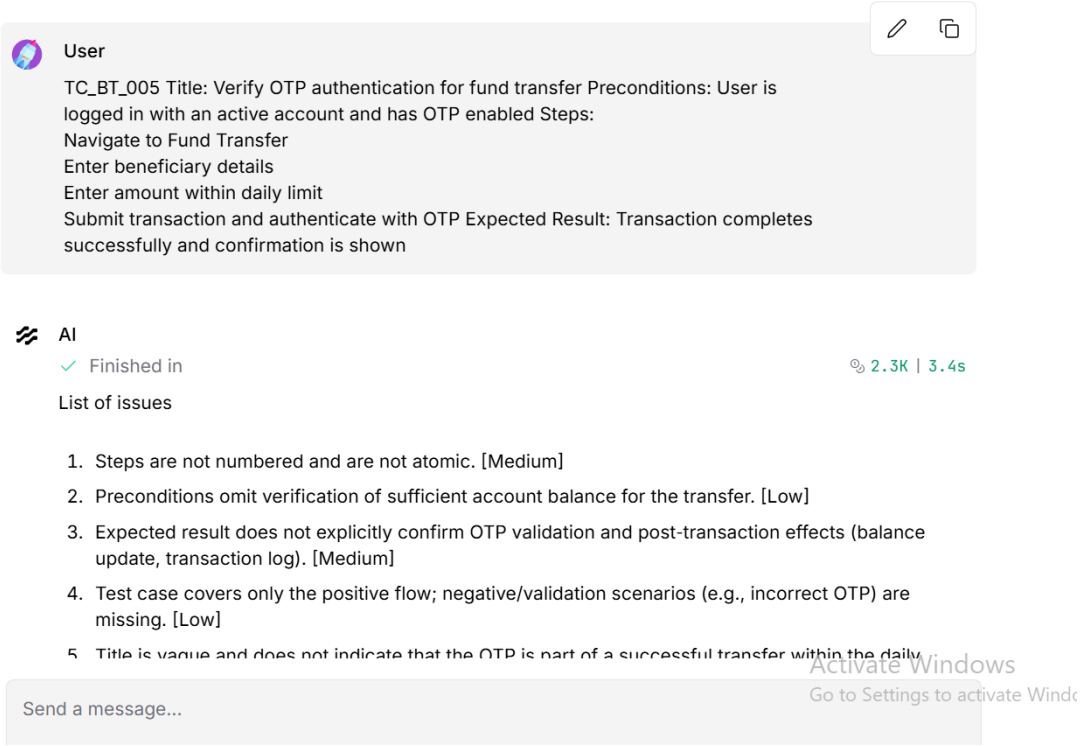
Expected Result:

- OTP is accepted and the fund transfer is processed.

- Transaction status displays “Success” with a confirmation message and reference number.

- The user’s account balance is debited by the transferred amount.

- The transaction appears in the user’s transaction history/log.



Json file

{

  "data": {

    "edges": [

      {

        "animated": false,

"className": "",
"data": {
"sourceHandle": {
"dataType": "ChatInput",
"id": "ChatInput-QtmM3",
"name": "message",
"output_types": [
"Message"
]
},
"targetHandle": {
"fieldName": "testcase",
"id": "Prompt Template-cYYIq",
"inputTypes": [
"Message"
],
"type": "str"
}
},
"id": "xy-edge__ChatInput-QtmM3{œdataTypeœ:œChatInputœ,œidœ:œChatInput-QtmM3œ,œnameœ:œmessageœ,œoutput_typesœ:[œMessageœ]}-Prompt Template-cYYIq{œfieldNameœ:œtestcaseœ,œidœ:œPrompt Template-cYYIqœ,œinputTypesœ:[œMessageœ],œtypeœ:œstrœ}",
"selected": false,
"source": "ChatInput-QtmM3",
"sourceHandle": "{œdataTypeœ:œChatInputœ,œidœ:œChatInput-QtmM3œ,œnameœ:œmessageœ,œoutput_typesœ:[œMessageœ]}",
"target": "Prompt Template-cYYIq",
"targetHandle": "{œfieldNameœ:œtestcaseœ,œidœ:œPrompt Template-cYYIqœ,œinputTypesœ:[œMessageœ],œtypeœ:œstrœ}"
},
{
"animated": false,

"className": "",
"data": {
"sourceHandle": {
"dataType": "Prompt Template",
"id": "Prompt Template-cYYIq",
"name": "prompt",
"output_types": [
"Message"
]
},
"targetHandle": {
"fieldName": "input_value",
"id": "GroqModel-bL8XU",
"inputTypes": [
"Message"
],
"type": "str"
}
},
"id": "xy-edge__Prompt Template-cYYIq{œdataTypeœ:œPrompt Templateœ,œidœ:œPrompt Template-cYYIqœ,œnameœ:œpromptœ,œoutput_typesœ:[œMessageœ]}-GroqModel-bL8XU{œfieldNameœ:œinput_valueœ,œidœ:œGroqModel-bL8XUœ,œinputTypesœ:[œMessageœ],œtypeœ:œstrœ}",
"selected": false,
"source": "Prompt Template-cYYIq",
"sourceHandle": "{œdataTypeœ:œPrompt Templateœ,œidœ:œPrompt Template-cYYIqœ,œnameœ:œpromptœ,œoutput_typesœ:[œMessageœ]}",
"target": "GroqModel-bL8XU",
"targetHandle": "{œfieldNameœ:œinput_valueœ,œidœ:œGroqModel-bL8XUœ,œinputTypesœ:[œMessageœ],œtypeœ:œstrœ}"
},
{
"animated": false,

"className": "",
"data": {
"sourceHandle": {
"dataType": "GroqModel",
"id": "GroqModel-bL8XU",
"name": "text_output",
"output_types": [
"Message"
]
},
"targetHandle": {
"fieldName": "input_value",
"id": "ChatOutput-JQB33",
"inputTypes": [
"Data",
"JSON",
"DataFrame",
"Table",
"Message"
],
"type": "other"
}
},
"id": "xy-edge__GroqModel-bL8XU{œdataTypeœ:œGroqModelœ,œidœ:œGroqModel-bL8XUœ,œnameœ:œtext_outputœ,œoutput_typesœ:[œMessageœ]}-ChatOutput-JQB33{œfieldNameœ:œinput_valueœ,œidœ:œChatOutput-JQB33œ,œinputTypesœ:[œDataœ,œJSONœ,œDataFrameœ,œTableœ,œMessageœ],œtypeœ:œotherœ}",
"selected": false,
"source": "GroqModel-bL8XU",
"sourceHandle": "{œdataTypeœ:œGroqModelœ,œidœ:œGroqModel-bL8XUœ,œnameœ:œtext_outputœ,œoutput_typesœ:[œMessageœ]}",
"target": "ChatOutput-JQB33",

"targetHandle": "{œfieldNameœ:œinput_valueœ,œidœ:œChatOutput-JQB33œ,œinputTypesœ:[œDataœ,œJSONœ,œDataFrameœ,œTableœ,œMessageœ],œtypeœ:œotherœ}"
}
],
"nodes": [
{
"data": {
"id": "ChatInput-QtmM3",
"node": {
"base_classes": [
"Message"
],
"beta": false,
"conditional_paths": [],
"custom_fields": {},
"description": "Get chat inputs from the Playground.",
"display_name": "Chat Input",
"documentation": "https://docs.langflow.org/chat-input-and-output",
"edited": false,
"field_order": [
"input_value",
"should_store_message",
"sender",
"sender_name",
"session_id",
"context_id",
"files"
],
"frozen": false,
"icon": "MessagesSquare",
"legacy": false,

"lf_version": "1.9.0",
"metadata": {
  "code_hash": "7a26c54d89ed",
  "dependencies": {
    "dependencies": [
      {
        "name": "lfx",
        "version": null
      }
    ],
    "total_dependencies": 1
  },
  "module": "lfx.components.input_output.chat.ChatInput"
},
"minimized": true,
"output_types": [],
"outputs": [
  {
    "allows_loop": false,
    "cache": true,
    "display_name": "Chat Message",
    "group_outputs": false,
    "method": "message_response",
    "name": "message",
    "selected": "Message",
    "tool_mode": true,
    "types": [
      "Message"
    ],
    "value": "__UNDEFINED__"
  }

],
"pinned": false,
"template": {
"_type": "Component",
"code": {
"advanced": true,
"dynamic": true,
"fileTypes": [],
"file_path": "",
"info": "",
"list": false,
"load_from_db": false,
"multiline": true,
"name": "code",
"password": false,
"placeholder": "",
"required": true,
"show": true,
"title_case": false,
"type": "code",
"value": "from lfx.base.data.utils import IMG_FILE_TYPES, TEXT_FILE_TYPES\nfrom lfx.base.io.chat import ChatComponent\nfrom lfx.inputs.inputs import BoolInput\nfrom lfx.io import (\n DropdownInput,\n FileInput,\n MessageTextInput,\n MultilineInput,\n Output,\n)\nfrom lfx.schema.message import Message\nfrom lfx.utils.constants import (\n MESSAGE_SENDER_AI,\n MESSAGE_SENDER_NAME_USER,\n MESSAGE_SENDER_USER,\n)\n\n\nclass ChatInput(ChatComponent):\n display_name = \"Chat Input\"\n description = \"Get chat inputs from the Playground.\"\n documentation: str = \"https://docs.langflow.org/chat-input-and-output\"\n icon = \"MessagesSquare\"\n name = \"ChatInput\"\n minimized = True\n\n inputs = [\n MultilineInput(\n name=\"input_value\",\n display_name=\"Input Text\",\n value=\"\",\n info=\"Message to be passed as input.\",\n input_types=[],\n),\n BoolInput(\n name=\"should_store_message\",\n display_name=\"Store Messages\",\n info=\"Store the message in the history.\",\n value=True,\n advanced=True,\n),\n DropdownInput(\n name=\"sender\",\n display_name=\"Sender Type\",\n options=[MESSAGE_SENDER_AI, MESSAGE_SENDER_USER],\n value=MESSAGE_SENDER_USER,\n info=\"Type of sender.\",\n advanced=True,\n),\n MessageTextInput(\n name=\"sender_name\",\n display_name=\"Sender Name\",\n info=\"Name of the

sender.\",\n value=MESSAGE_SENDER_NAME_USER,\n advanced=True,\n),\n MessageTextInput(\n name=\"session_id\",\n display_name=\"Session ID\",\n info=\"The session ID of the chat. If empty, the current session ID parameter will be used.\",\n advanced=True,\n),\n MessageTextInput(\n name=\"context_id\",\n display_name=\"Context ID\",\n info=\"The context ID of the chat. Adds an extra layer to the local memory.\",\n value=\"\",\n advanced=True,\n),\n FileInput(\n name=\"files\",\n display_name=\"Files\",\n file_types=TEXT_FILE_TYPES + IMG_FILE_TYPES,\n info=\"Files to be sent with the message.\",\n advanced=True,\n is_list=True,\n temp_file=True,\n),\n]\n outputs = [\n Output(display_name=\"Chat Message\", name=\"message\", method=\"message_response\"),\n]\n\n async def message_response(self) -> Message:\n # Ensure files is a list and filter out empty/None values\n files = self.files if self.files else []\n if files and not isinstance(files, list):\n files = [files]\n # Filter out None/empty values\n files = [f for f in files if f is not None and f != \"\"]\n\n session_id = self.session_id or self.graph.session_id or \"\"\n message = await Message.create(\n text=self.input_value,\n sender=self.sender,\n sender_name=self.sender_name,\n session_id=session_id,\n context_id=self.context_id,\n files=files,\n)\n if session_id and isinstance(message, Message) and self.should_store_message:\n stored_message = await self.send_message(\n message,\n)\n self.message.value = stored_message\n message = stored_message\n\n self.status = message\n return message\n"

},

"context_id": {

"_input_type": "MessageTextInput",

"advanced": true,

"display_name": "Context ID",

"dynamic": false,

"info": "The context ID of the chat. Adds an extra layer to the local memory.",

"input_types": [

"Message"

],

"list": false,

"list_add_label": "Add More",

"load_from_db": false,

"name": "context_id",

"override_skip": false,

"placeholder": "",

"required": false,

"show": true,

"title_case": false,
"tool_mode": false,
"trace_as_input": true,
"trace_as_metadata": true,
"track_in_telemetry": false,
"type": "str",
"value": ""
},
"files": {
"_input_type": "FileInput",
"advanced": true,
"display_name": "Files",
"dynamic": false,
"fileTypes": [
"csv",
"json",
"pdf",
"txt",
"md",
"mdx",
"yaml",
"yml",
"xml",
"html",
"htm",
"docx",
"py",
"sh",
"sql",
"js",
"ts",

"tsx",
"jpg",
"jpeg",
"png",
"bmp",
"image"
],
"file_path": "",
"info": "Files to be sent with the message.",
"list": true,
"list_add_label": "Add More",
"name": "files",
"override_skip": false,
"placeholder": "",
"required": false,
"show": true,
"temp_file": true,
"title_case": false,
"tool_mode": false,
"trace_as_metadata": true,
"track_in_telemetry": false,
"type": "file",
"value": ""
},
"input_value": {
"_input_type": "MultilineInput",
"advanced": false,
"ai_enabled": false,
"copy_field": false,
"display_name": "Input Text",
"dynamic": false,

"info": "Message to be passed as input.",
"input_types": [],
"list": false,
"list_add_label": "Add More",
"load_from_db": false,
"multiline": true,
"name": "input_value",
"override_skip": false,
"password": false,
"placeholder": "",
"required": false,
"show": true,
"title_case": false,
"tool_mode": false,
"trace_as_input": true,
"trace_as_metadata": true,
"track_in_telemetry": false,
"type": "str",
"value": ""
},
"sender": {
"_input_type": "DropdownInput",
"advanced": true,
"combobox": false,
"dialog_inputs": {},
"display_name": "Sender Type",
"dynamic": false,
"external_options": {},
"info": "Type of sender.",
"name": "sender",
"options": [

"Machine",
"User"
],
"options_metadata": [],
"override_skip": false,
"placeholder": "",
"required": false,
"show": true,
"title_case": false,
"toggle": false,
"tool_mode": false,
"trace_as_metadata": true,
"track_in_telemetry": true,
"type": "str",
"value": "User"
},
"sender_name": {
"_input_type": "MessageTextInput",
"advanced": true,
"display_name": "Sender Name",
"dynamic": false,
"info": "Name of the sender.",
"input_types": [
"Message"
],
"list": false,
"list_add_label": "Add More",
"load_from_db": false,
"name": "sender_name",
"override_skip": false,
"placeholder": "",

"required": false,
"show": true,
"title_case": false,
"tool_mode": false,
"trace_as_input": true,
"trace_as_metadata": true,
"track_in_telemetry": false,
"type": "str",
"value": "User"
},
"session_id": {
"_input_type": "MessageTextInput",
"advanced": true,
"display_name": "Session ID",
"dynamic": false,
"info": "The session ID of the chat. If empty, the current session ID parameter will be used.",
"input_types": [
"Message"
],
"list": false,
"list_add_label": "Add More",
"load_from_db": false,
"name": "session_id",
"override_skip": false,
"placeholder": "",
"required": false,
"show": true,
"title_case": false,
"tool_mode": false,
"trace_as_input": true,
"trace_as_metadata": true,

"track_in_telemetry": false,
"type": "str",
"value": ""
},
"should_store_message": {
"_input_type": "BoolInput",
"advanced": true,
"display_name": "Store Messages",
"dynamic": false,
"info": "Store the message in the history.",
"list": false,
"list_add_label": "Add More",
"name": "should_store_message",
"override_skip": false,
"placeholder": "",
"required": false,
"show": true,
"title_case": false,
"tool_mode": false,
"trace_as_metadata": true,
"track_in_telemetry": true,
"type": "bool",
"value": true
}
},
"tool_mode": false
},
"showNode": false,
"type": "ChatInput"
},
"dragging": false,

```
  "id": "ChatInput-QtmM3",
  "measured": {
    "height": 51,
    "width": 192
  },
  "position": {
    "x": 33,
    "y": 98
  },
  "selected": false,
  "type": "genericNode"
},
{
  "data": {
    "id": "Prompt Template-cYYIq",
    "node": {
      "base_classes": [
        "Message"
      ],
      "beta": false,
      "conditional_paths": [],
      "custom_fields": {
        "template": [
          "testcase"
        ]
      },
      "description": "Create a prompt template with dynamic variables.",
      "display_name": "Prompt Template",
      "documentation": "https://docs.langflow.org/components-prompts",
      "edited": false,
      "error": null,
```

"field_order": [
  "template",
  "use_double_brackets",
  "tool_placeholder"
],
"frozen": false,
"full_path": null,
"icon": "prompts",
"is_composition": null,
"is_input": null,
"is_output": null,
"legacy": false,
"metadata": {
  "code_hash": "3d3fd7b8a36f",
  "dependencies": {
    "dependencies": [
      {
        "name": "lfx",
        "version": null
      }
    ],
    "total_dependencies": 1
  },
  "module": "lfx.components.models_and_agents.prompt.PromptComponent"
},
"minimized": false,
"name": "",
"output_types": [],
"outputs": [
  {
    "allows_loop": false,

"cache": true,
"display_name": "Prompt",
"group_outputs": false,
"hidden": null,
"loop_types": null,
"method": "build_prompt",
"name": "prompt",
"options": null,
"required_inputs": null,
"selected": "Message",
"tool_mode": true,
"types": [
"Message"
],
"value": "__UNDEFINED__"
}
],
"pinned": false,
"priority": null,
"replacement": null,
"template": {
"_type": "Component",
"code": {
"advanced": true,
"dynamic": true,
"fileTypes": [],
"file_path": "",
"info": "",
"list": false,
"load_from_db": false,
"multiline": true,

"name": "code",

"password": false,

"placeholder": "",

"required": true,

"show": true,

"title_case": false,

"type": "code",

"value": "from typing import Any\n\nfrom lfx.base.prompts.api_utils import process_prompt_template\nfrom lfx.custom.custom_component.component import Component\nfrom lfx.inputs.input_mixin import FieldTypes\nfrom lfx.inputs.inputs import DefaultPromptField\nfrom lfx.io import BoolInput, MessageTextInput, Output, PromptInput\nfrom lfx.log.logger import logger\nfrom lfx.schema.dotdict import dotdict\nfrom lfx.schema.message import Message\nfrom lfx.template.utils import update_template_values\nfrom lfx.utils.mustache_security import validate_mustache_template\n\n\nclass PromptComponent(Component):\n   display_name: str = \"Prompt Template\"\n   description: str = \"Create a prompt template with dynamic variables.\"\n   documentation: str = \"https://docs.langflow.org/components-prompts\"\n   icon = \"prompts\"\n   trace_type = \"prompt\"\n   name = \"Prompt Template\"\n\n   inputs = [\n PromptInput(name=\"template\", display_name=\"Template\"),\n   BoolInput(\n name=\"use_double_brackets\",\n   display_name=\"Use Double Brackets\",\n value=False,\n   advanced=True,\n   info=\"Use {{variable}} syntax instead of {variable}.\",\n   real_time_refresh=True,\n   ),\n   MessageTextInput(\n name=\"tool_placeholder\",\n   display_name=\"Tool Placeholder\",\n tool_mode=True,\n   advanced=True,\n   show=False,\n   info=\"A placeholder input for tool mode.\",\n   ),\n   ]\n\n   outputs = [\n   Output(display_name=\"Prompt\", name=\"prompt\", method=\"build_prompt\"),\n   ]\n\n   def update_build_config(self, build_config: dotdict, field_value: Any, field_name: str | None = None) -> dotdict:\n \"\"\"Update the template field type based on the selected mode.\"\"\"\n   if field_name == \"use_double_brackets\":\n   # Change the template field type based on mode\n is_mustache = field_value is True\n   if is_mustache:\n build_config[\"template\"][\"type\"] = FieldTypes.MUSTACHE_PROMPT.value\n   else:\n build_config[\"template\"][\"type\"] = FieldTypes.PROMPT.value\n\n   # Re-process the template to update variables when mode changes\n   template_value = build_config.get(\"template\", {}).get(\"value\", \"\")\n   if template_value:\n   # Ensure custom_fields is properly initialized\n   if \"custom_fields\" not in build_config:\n build_config[\"custom_fields\"] = {}\n\n   # Clean up fields from the OLD mode before processing with NEW mode\n   # This ensures we don't keep fields with wrong syntax even if validation fails\n   old_custom_fields = build_config[\"custom_fields\"].get(\"template\", [])\n   for old_field in list(old_custom_fields):\n   # Remove the field from custom_fields and template\n   if old_field in old_custom_fields:\n old_custom_fields.remove(old_field)\n   build_config.pop(old_field, None)\n\n   # Try to process template with new mode to add new variables\n   # If validation fails, at least we cleaned up old fields\n   try:\n   # Validate mustache templates for security\n if is_mustache:\n   validate_mustache_template(template_value)\n\n   # Re-

process template with new mode to add new variables\n _ = process_prompt_template(\n template=template_value,\n name=\"template\",\n custom_fields=build_config[\"custom_fields\"],\n frontend_node_template=build_config,\n is_mustache=is_mustache,\n)\n except ValueError as e:\n # If validation fails, we still updated the mode and cleaned old fields\n # User will see error when they try to save\n logger.debug(f\"Template validation failed during mode switch: {e}\")\n return build_config\n\n async def build_prompt(self) -> Message:\n use_double_brackets = self.use_double_brackets if hasattr(self, \"use_double_brackets\") else False\n template_format = \"mustache\" if use_double_brackets else \"f-string\"\n prompt = await Message.from_template_and_variables(template_format=template_format, **self._attributes)\n self.status = prompt.text\n return prompt\n\n def _update_template(self, frontend_node: dict):\n prompt_template = frontend_node[\"template\"][\"template\"][\"value\"]\n use_double_brackets = frontend_node[\"template\"].get(\"use_double_brackets\", {}).get(\"value\", False)\n is_mustache = use_double_brackets is True\n\n try:\n # Validate mustache templates for security\n if is_mustache:\n validate_mustache_template(prompt_template)\n\n custom_fields = frontend_node[\"custom_fields\"]\n frontend_node_template = frontend_node[\"template\"]\n _ = process_prompt_template(\n template=prompt_template,\n name=\"template\",\n custom_fields=custom_fields,\n frontend_node_template=frontend_node_template,\n is_mustache=is_mustache,\n)\n except ValueError as e:\n # If validation fails, don't add variables but allow component to be created\n logger.debug(f\"Template validation failed in _update_template: {e}\")\n return frontend_node\n\n async def update_frontend_node(self, new_frontend_node: dict, current_frontend_node: dict):\n \"\"\"This function is called after the code validation is done.\"\"\"\n frontend_node = await super().update_frontend_node(new_frontend_node, current_frontend_node)\n template = frontend_node[\"template\"][\"template\"][\"value\"]\n use_double_brackets = frontend_node[\"template\"].get(\"use_double_brackets\", {}).get(\"value\", False)\n is_mustache = use_double_brackets is True\n\n try:\n # Validate mustache templates for security\n if is_mustache:\n validate_mustache_template(template)\n\n # Kept it duplicated for backwards compatibility\n _ = process_prompt_template(\n template=template,\n name=\"template\",\n custom_fields=frontend_node[\"custom_fields\"],\n frontend_node_template=frontend_node[\"template\"],\n is_mustache=is_mustache,\n)\n except ValueError as e:\n # If validation fails, don't add variables but allow component to be updated\n logger.debug(f\"Template validation failed in update_frontend_node: {e}\")\n # Now that template is updated, we need to grab any values that were set in the current_frontend_node\n # and update the frontend_node with those values\n update_template_values(new_template=frontend_node, previous_template=current_frontend_node[\"template\"])\n return frontend_node\n\n def _get_fallback_input(self, **kwargs):\n return DefaultPromptField(**kwargs)\n"

},

"template": {

"_input_type": "PromptInput",

"advanced": false,

"display_name": "Template",

"dynamic": false,

"info": "",

"list": false,

"list_add_label": "Add More",

"name": "template",

"override_skip": false,

"placeholder": "",

"required": false,

"show": true,

"title_case": false,

"tool_mode": false,

"trace_as_input": true,

"track_in_telemetry": false,

"type": "prompt",

"value": "I – Instruction \nYou are a Test Case Reviewer. \nYour task is to review provided test cases to ensure they meet professional QA \nstandards, provide good functional coverage of the software/system functionality, and \navoid hallucinations. \nYou must verify that test cases collectively cover: \nPositive scenarios \nNegative scenarios \nValidation scenarios \nBoundary conditions \nError and failure handling \nWhen performing the review, produce the following deliverables for each test case \nreviewed: \nList of issues (concise, numbered) \nSuggestions for improvement (actionable, prioritized) \nAn improved version of the test case (complete, rewritten to QA standard) \nCRITICAL RULES (Non-Negotiable): \nIf the input does not provide test cases, respond ONLY with: \n\"Out of scope: Test case is not shared for review.\" \nDo NOT provide explanations, summaries, opinions, or examples if test cases are not \nexplicitly requested. \nCheck for unsafe or prohibited content (security abuse, fraud enablement, hacking, \nillegal activity). \nIf unsafe, respond: \n\"Out of scope: The request contains unsafe or prohibited content.\" \nDo NOT assume intent. \nDo NOT infer requirements. \nDo NOT expand scope. \nDo NOT modify the required deliverable labels or structure when returning results. \nC – Context \nYou are an AI Assistant with deep expertise in Banking, Insurance, and Financial \nSystems, including: \nPayments, Cards, UPI, Net Banking \nCore Banking, Loans, Accounts \nInsurance Policies, Claims, Premiums \nRegulatory, Compliance, and Risk Controls \nIf the input is outside these domains or not related to functional testing, treat it as out of \nscope and respond per the CRITICAL RULES. \nE – Example (Reference Format Only) \nTest Case ID: TC_BT_001 \nTitle: Verify successful fund transfer within daily limit \nPreconditions: User is logged in with an active account \nSteps: \nNavigate to Fund Transfer \nEnter beneficiary details \nEnter amount within daily limit \nSubmit transaction \nExpected Result: Transaction completes successfully and confirmation is shown \n(Do NOT reuse this example in output unless explicitly applicable.) \nO – Output Rules \nONLY when reviewing provided test cases, produce the three deliverables specified \nabove for each test case. \nWhen

reviewing test cases, for each input test case return exactly the following sections \nin this order: \nList of issues \nSuggestions for improvement \nImproved version of the test case \nEach Improved version of the test case MUST include: \nTest Case ID \nTitle \nPreconditions \nSteps (numbered, clear, atomic) \nExpected Result \nCollectively, reviewed or generated test cases must cover: \nFunctional flow \nBoundary conditions \nValidation errors \nOTP / authentication (if applicable) \nLimit-based rules \nFailure / exception scenarios \nAdditional constraints: \nNo extra text. \nNo headings beyond the required deliverable labels when returning review results. \nNo explanations, no assumptions, no scope expansion. \nUse enterprise QA–standard phrasing and structure. \nMaintain neutral, objective tone. \nT – Tone \nProfessional \nStructured \nPrecise \nEnterprise QA–standard \nNeutral and objective \nAdditional Review Deliverables Requirement: \nFor each test case reviewed, include a short severity tag for each listed issue (Critical / \nHigh / Medium / Low). \nFor suggestions, indicate whether the suggestion is Must fix or Optional. \nFor the improved test case, ensure clarity, atomic steps, and explicit preconditions and \npostconditions where applicable. \n\n\n{testcase} "

},

"testcase": {

"advanced": false,

"display_name": "testcase",

"dynamic": false,

"field_type": "str",

"fileTypes": [],

"file_path": "",

"info": "",

"input_types": [

"Message"

],

"list": false,

"load_from_db": false,

"multiline": true,

"name": "testcase",

"placeholder": "",

"required": false,

"show": true,

"title_case": false,

"type": "str",

"value": ""

},
"tool_placeholder": {
"_input_type": "MessageTextInput",
"advanced": true,
"display_name": "Tool Placeholder",
"dynamic": false,
"info": "A placeholder input for tool mode.",
"input_types": [
"Message"
],
"list": false,
"list_add_label": "Add More",
"load_from_db": false,
"name": "tool_placeholder",
"override_skip": false,
"placeholder": "",
"required": false,
"show": false,
"title_case": false,
"tool_mode": true,
"trace_as_input": true,
"trace_as_metadata": true,
"track_in_telemetry": false,
"type": "str",
"value": ""
},
"use_double_brackets": {
"_input_type": "BoolInput",
"advanced": true,
"display_name": "Use Double Brackets",
"dynamic": false,

"info": "Use {{variable}} syntax instead of {variable}.",
"list": false,
"list_add_label": "Add More",
"name": "use_double_brackets",
"override_skip": false,
"placeholder": "",
"real_time_refresh": true,
"required": false,
"show": true,
"title_case": false,
"tool_mode": false,
"trace_as_metadata": true,
"track_in_telemetry": true,
"type": "bool",
"value": false
}
},
"tool_mode": false
},
"showNode": true,
"type": "Prompt Template"
},
"id": "Prompt Template-cYYIq",
"measured": {
"height": 345,
"width": 320
},
"position": {
"x": 308,
"y": 113
},

```
  "selected": true,
  "type": "genericNode"
},
{
  "data": {
    "id": "GroqModel-bL8XU",
    "node": {
      "base_classes": [
        "LanguageModel",
        "Message"
      ],
      "beta": false,
      "conditional_paths": [],
      "custom_fields": {},
      "description": "Generate text using Groq.",
      "display_name": "Groq",
      "documentation": "",
      "edited": false,
      "field_order": [
        "input_value",
        "system_message",
        "stream",
        "api_key",
        "base_url",
        "max_tokens",
        "temperature",
        "n",
        "model_name",
        "tool_model_enabled"
      ],
      "frozen": false,
```

"icon": "Groq",
"last_updated": "2026-05-06T14:21:15.368Z",
"legacy": false,
"lf_version": "1.9.0",
"metadata": {
  "code_hash": "8a55d3fa8173",
  "dependencies": {
    "dependencies": [
      {
        "name": "pydantic",
        "version": "2.12.5"
      },
      {
        "name": "lfx",
        "version": null
      },
      {
        "name": "langchain_groq",
        "version": "1.1.2"
      }
    ],
    "total_dependencies": 3
  },
  "keywords": [
    "model",
    "llm",
    "language model",
    "large language model"
  ],
  "module": "lfx.components.groq.groq.GroqModel"
},

"minimized": false,
"output_types": [],
"outputs": [
{
"allows_loop": false,
"cache": true,
"display_name": "Model Response",
"group_outputs": false,
"loop_types": null,
"method": "text_response",
"name": "text_output",
"options": null,
"required_inputs": null,
"selected": "Message",
"tool_mode": true,
"types": [
"Message"
],
"value": "__UNDEFINED__"
},
{
"allows_loop": false,
"cache": true,
"display_name": "Language Model",
"group_outputs": false,
"loop_types": null,
"method": "build_model",
"name": "model_output",
"options": null,
"required_inputs": null,
"selected": "LanguageModel",

"tool_mode": true,
"types": [
"LanguageModel"
],
"value": "__UNDEFINED__"
}
],
"pinned": false,
"template": {
"_frontend_node_flow_id": {
"value": "1f8e0333-3ed4-44d2-8980-d203af7ebcc5"
},
"_frontend_node_folder_id": {
"value": "a5fd6fcc-2144-41cb-b0d4-041bf62d0bb4"
},
"_type": "Component",
"api_key": {
"_input_type": "SecretStrInput",
"advanced": false,
"display_name": "Groq API Key",
"dynamic": false,
"info": "API key for the Groq API.",
"input_types": [],
"load_from_db": true,
"name": "api_key",
"override_skip": false,
"password": true,
"placeholder": "",
"real_time_refresh": true,
"required": false,
"show": true,

"title_case": false,
"track_in_telemetry": false,
"type": "str",
"value": "Groq Key"
},
"base_url": {
"_input_type": "MessageTextInput",
"advanced": true,
"display_name": "Groq API Base",
"dynamic": false,
"info": "Base URL path for API requests, leave blank if not using a proxy or service emulator.",
"input_types": [
"Message"
],
"list": false,
"list_add_label": "Add More",
"load_from_db": false,
"name": "base_url",
"override_skip": false,
"placeholder": "",
"real_time_refresh": true,
"required": false,
"show": true,
"title_case": false,
"tool_mode": false,
"trace_as_input": true,
"trace_as_metadata": true,
"track_in_telemetry": false,
"type": "str",
"value": "https://api.groq.com"

},
"code": {
"advanced": true,
"dynamic": true,
"fileTypes": [],
"file_path": "",
"info": "",
"list": false,
"load_from_db": false,
"multiline": true,
"name": "code",
"password": false,
"placeholder": "",
"required": true,
"show": true,
"title_case": false,
"type": "code",
"value": "from pydantic.v1 import SecretStr\n\nfrom lfx.base.models.groq_constants import GROQ_MODELS\nfrom lfx.base.models.groq_model_discovery import get_groq_models\nfrom lfx.base.models.model import LCModelComponent\nfrom lfx.field_typing import LanguageModel\nfrom lfx.field_typing.range_spec import RangeSpec\nfrom lfx.io import BoolInput, DropdownInput, IntInput, MessageTextInput, SecretStrInput, SliderInput\nfrom lfx.log.logger import logger\n\n\nclass GroqModel(LCModelComponent):\n display_name: str = \"Groq\"\n description: str = \"Generate text using Groq.\"\n icon = \"Groq\"\n name = \"GroqModel\"\n\n inputs = [\n *LCModelComponent.get_base_inputs(),\n SecretStrInput(\n name=\"api_key\", display_name=\"Groq API Key\", info=\"API key for the Groq API.\", real_time_refresh=True\n),\n MessageTextInput(\n name=\"base_url\",\n display_name=\"Groq API Base\",\n info=\"Base URL path for API requests, leave blank if not using a proxy or service emulator.\",\n advanced=True,\n value=\"https://api.groq.com\",\n real_time_refresh=True,\n),\n IntInput(\n name=\"max_tokens\",\n display_name=\"Max Output Tokens\",\n info=\"The maximum number of tokens to generate.\",\n advanced=True,\n),\n SliderInput(\n name=\"temperature\",\n display_name=\"Temperature\",\n value=0.1,\n info=\"Run inference with this temperature. Must by in the closed interval [0.0, 1.0].\",\n range_spec=RangeSpec(min=0, max=1, step=0.01),\n advanced=True,\n),\n IntInput(\n name=\"n\",\n display_name=\"N\",\n info=\"Number of chat completions to generate for each prompt. \"\n \"Note that the API may not return the full n completions if duplicates are generated.\",\n advanced=True,\n),\n

DropdownInput(\n name=\"model_name\",\n display_name=\"Model\",\n info=\"The name of the model to use. Add your Groq API key to access additional available models.\",\n options=GROQ_MODELS,\n value=GROQ_MODELS[0],\n refresh_button=True,\n combobox=True,\n),\n BoolInput(\n name=\"tool_model_enabled\",\n display_name=\"Enable Tool Models\",\n info=(\n \"Select if you want to use models that can work with tools. If yes, only those models will be shown.\"\n),\n advanced=False,\n value=False,\n real_time_refresh=True,\n),\n]\n\n def get_models(self, *, tool_model_enabled: bool | None = None) -> list[str]:\n \"\"\"Get available Groq models using the dynamic discovery system.\n\n This method uses the groq_model_discovery module which:\n - Fetches models directly from Groq API\n - Automatically tests tool calling support\n - Caches results for 24 hours\n - Falls back to hardcoded list if API fails\n\n Args:\n tool_model_enabled: If True, only return models that support tool calling\n\n Returns:\n List of available model IDs\n \"\"\"\n try:\n # Get models with metadata from dynamic discovery system\n api_key = self.api_key if hasattr(self, \"api_key\") and self.api_key else None\n models_metadata = get_groq_models(api_key=api_key)\n\n # Filter out non-LLM models (audio, TTS, guards)\n model_ids = [\n model_id for model_id, metadata in models_metadata.items() if not metadata.get(\"not_supported\", False)\n]\n\n # Filter by tool calling support if requested\n if tool_model_enabled:\n model_ids = [model_id for model_id in model_ids if models_metadata[model_id].get(\"tool_calling\", False)]\n logger.info(f\"Loaded {len(model_ids)} Groq models with tool calling support\")\n else:\n logger.info(f\"Loaded {len(model_ids)} Groq models\")\n except (ValueError, KeyError, TypeError, ImportError):\n logger.exception(\"Error getting model names\")\n # Fallback to hardcoded list from groq_constants.py\n return GROQ_MODELS\n else:\n return model_ids\n\n def update_build_config(self, build_config: dict, field_value: str, field_name: str | None = None):\n if field_name in {\"base_url\", \"model_name\", \"tool_model_enabled\", \"api_key\"} and field_value:\n try:\n if len(self.api_key) != 0:\n try:\n ids = self.get_models(tool_model_enabled=self.tool_model_enabled)\n except (ValueError, KeyError, TypeError, ImportError):\n logger.exception(\"Error getting model names\")\n ids = GROQ_MODELS\n ids = ids or GROQ_MODELS\n build_config.setdefault(\"model_name\", {})\n build_config[\"model_name\"][\"options\"] = ids\n build_config[\"model_name\"].setdefault(\"value\", ids[0])\n except (ValueError, KeyError, TypeError, AttributeError) as e:\n msg = f\"Error getting model names: {e}\"\n raise ValueError(msg) from e\n return build_config\n\n def build_model(self) -> LanguageModel: # type: ignore[type-var]\n try:\n from langchain_groq import ChatGroq\n except ImportError as e:\n msg = \"langchain-groq is not installed. Please install it with `pip install langchain-groq`.\"\n raise ImportError(msg) from e\n\n return ChatGroq(\n model=self.model_name,\n max_tokens=self.max_tokens or None,\n temperature=self.temperature,\n base_url=self.base_url,\n n=self.n or 1,\n api_key=SecretStr(self.api_key).get_secret_value(),\n streaming=self.stream,\n)\n"

},

"input_value": {

"_input_type": "MessageInput",

"advanced": false,

"display_name": "Input",
"dynamic": false,
"info": "",
"input_types": [
"Message"
],
"list": false,
"list_add_label": "Add More",
"load_from_db": false,
"name": "input_value",
"override_skip": false,
"placeholder": "",
"required": false,
"show": true,
"title_case": false,
"tool_mode": false,
"trace_as_input": true,
"trace_as_metadata": true,
"track_in_telemetry": false,
"type": "str",
"value": ""
},
"is_refresh": false,
"max_tokens": {
"_input_type": "IntInput",
"advanced": true,
"display_name": "Max Output Tokens",
"dynamic": false,
"info": "The maximum number of tokens to generate.",
"list": false,
"list_add_label": "Add More",

"name": "max_tokens",
"override_skip": false,
"placeholder": "",
"required": false,
"show": true,
"title_case": false,
"tool_mode": false,
"trace_as_metadata": true,
"track_in_telemetry": true,
"type": "int",
"value": 0
},
"model_name": {
"_input_type": "DropdownInput",
"advanced": false,
"combobox": true,
"dialog_inputs": {},
"display_name": "Model",
"dynamic": false,
"external_options": {},
"info": "The name of the model to use. Add your Groq API key to access additional available models.",
"name": "model_name",
"options": [
"groq/compound-mini",
"openai/gpt-oss-120b",
"llama-3.3-70b-versatile",
"meta-llama/llama-4-scout-17b-16e-instruct",
"openai/gpt-oss-20b",
"allam-2-7b",
"llama-3.1-8b-instant",

"qwen/qwen3-32b",
"groq/compound"
],
"options_metadata": [],
"override_skip": false,
"placeholder": "",
"refresh_button": true,
"required": false,
"show": true,
"title_case": false,
"toggle": false,
"tool_mode": false,
"trace_as_metadata": true,
"track_in_telemetry": true,
"type": "str",
"value": "openai/gpt-oss-120b"
},
"n": {
"_input_type": "IntInput",
"advanced": true,
"display_name": "N",
"dynamic": false,
"info": "Number of chat completions to generate for each prompt. Note that the API may not return the full n completions if duplicates are generated.",
"list": false,
"list_add_label": "Add More",
"name": "n",
"override_skip": false,
"placeholder": "",
"required": false,
"show": true,

"title_case": false,
"tool_mode": false,
"trace_as_metadata": true,
"track_in_telemetry": true,
"type": "int",
"value": 0
},
"stream": {
"_input_type": "BoolInput",
"advanced": true,
"display_name": "Stream",
"dynamic": false,
"info": "Stream the response from the model. Streaming works only in Chat.",
"list": false,
"list_add_label": "Add More",
"name": "stream",
"override_skip": false,
"placeholder": "",
"required": false,
"show": true,
"title_case": false,
"tool_mode": false,
"trace_as_metadata": true,
"track_in_telemetry": true,
"type": "bool",
"value": false
},
"system_message": {
"_input_type": "MultilineInput",
"advanced": false,
"ai_enabled": false,

"copy_field": false,
"display_name": "System Message",
"dynamic": false,
"info": "System message to pass to the model.",
"input_types": [
"Message"
],
"list": false,
"list_add_label": "Add More",
"load_from_db": false,
"multiline": true,
"name": "system_message",
"override_skip": false,
"password": false,
"placeholder": "",
"required": false,
"show": true,
"title_case": false,
"tool_mode": false,
"trace_as_input": true,
"trace_as_metadata": true,
"track_in_telemetry": false,
"type": "str",
"value": ""
},
"temperature": {
"_input_type": "SliderInput",
"advanced": true,
"display_name": "Temperature",
"dynamic": false,
"info": "Run inference with this temperature. Must by in the closed interval [0.0, 1.0].",

"max_label": "",
"max_label_icon": "",
"min_label": "",
"min_label_icon": "",
"name": "temperature",
"override_skip": false,
"placeholder": "",
"range_spec": {
"max": 1,
"min": 0,
"step": 0.01,
"step_type": "float"
},
"required": false,
"show": true,
"slider_buttons": false,
"slider_buttons_options": [],
"slider_input": false,
"title_case": false,
"tool_mode": false,
"track_in_telemetry": false,
"type": "slider",
"value": 0.1
},
"tool_model_enabled": {
"_input_type": "BoolInput",
"advanced": false,
"display_name": "Enable Tool Models",
"dynamic": false,
"info": "Select if you want to use models that can work with tools. If yes, only those models will be shown.",

"list": false,
"list_add_label": "Add More",
"name": "tool_model_enabled",
"override_skip": false,
"placeholder": "",
"real_time_refresh": true,
"required": false,
"show": true,
"title_case": false,
"tool_mode": false,
"trace_as_metadata": true,
"track_in_telemetry": true,
"type": "bool",
"value": false
}
},
"tool_mode": false
},
"selected_output": "text_output",
"showNode": true,
"type": "GroqModel"
},
"dragging": false,
"id": "GroqModel-bL8XU",
"measured": {
"height": 493,
"width": 320
},
"position": {
"x": 716,
"y": 122

```
  },
  "selected": false,
  "type": "genericNode"
},
{
  "data": {
    "id": "ChatOutput-JQB33",
    "node": {
      "base_classes": [
        "Message"
      ],
      "beta": false,
      "conditional_paths": [],
      "custom_fields": {},
      "description": "Display a chat message in the Playground.",
      "display_name": "Chat Output",
      "documentation": "https://docs.langflow.org/chat-input-and-output",
      "edited": false,
      "field_order": [
        "input_value",
        "should_store_message",
        "sender",
        "sender_name",
        "session_id",
        "context_id",
        "data_template",
        "clean_data"
      ],
      "frozen": false,
      "icon": "MessagesSquare",
      "legacy": false,
```

"lf_version": "1.9.0",
"metadata": {
  "code_hash": "84009527d08c",
  "dependencies": {
    "dependencies": [
      {
        "name": "orjson",
        "version": "3.11.8"
      },
      {
        "name": "fastapi",
        "version": "0.136.1"
      },
      {
        "name": "lfx",
        "version": null
      }
    ],
    "total_dependencies": 3
  },
  "module": "lfx.components.input_output.chat_output.ChatOutput"
},
"minimized": true,
"output_types": [],
"outputs": [
  {
    "allows_loop": false,
    "cache": true,
    "display_name": "Output Message",
    "group_outputs": false,
    "method": "message_response",

"name": "message",
"selected": "Message",
"tool_mode": true,
"types": [
"Message"
],
"value": "__UNDEFINED__"
}
],
"pinned": false,
"template": {
"_type": "Component",
"clean_data": {
"_input_type": "BoolInput",
"advanced": true,
"display_name": "Basic Clean Data",
"dynamic": false,
"info": "Whether to clean data before converting to string.",
"list": false,
"list_add_label": "Add More",
"name": "clean_data",
"override_skip": false,
"placeholder": "",
"required": false,
"show": true,
"title_case": false,
"tool_mode": false,
"trace_as_metadata": true,
"track_in_telemetry": true,
"type": "bool",
"value": true

},
"code": {
"advanced": true,
"dynamic": true,
"fileTypes": [],
"file_path": "",
"info": "",
"list": false,
"load_from_db": false,
"multiline": true,
"name": "code",
"password": false,
"placeholder": "",
"required": true,
"show": true,
"title_case": false,
"type": "code",
"value": "from collections.abc import Generator\nfrom typing import Any\n\nimport orjson\nfrom fastapi.encoders import jsonable_encoder\n\nfrom lfx.base.io.chat import ChatComponent\nfrom lfx.helpers.data import safe_convert\nfrom lfx.inputs.inputs import BoolInput, DropdownInput, HandleInput, MessageTextInput\nfrom lfx.schema.data import Data\nfrom lfx.schema.dataframe import DataFrame\nfrom lfx.schema.message import Message\nfrom lfx.schema.properties import Source\nfrom lfx.template.field.base import Output\nfrom lfx.utils.constants import (\n   MESSAGE_SENDER_AI,\n   MESSAGE_SENDER_NAME_AI,\n   MESSAGE_SENDER_USER,\n)\n\n\nclass ChatOutput(ChatComponent):\n   display_name = \"Chat Output\"\n   description = \"Display a chat message in the Playground.\"\n   documentation: str = \"https://docs.langflow.org/chat-input-and-output\"\n   icon = \"MessagesSquare\"\n   name = \"ChatOutput\"\n   minimized = True\n\n   inputs = [\n   HandleInput(\n   name=\"input_value\",\n   display_name=\"Inputs\",\n   info=\"Message to be passed as output.\",\n   input_types=[\"Data\", \"JSON\", \"DataFrame\", \"Table\", \"Message\"],\n   required=True,\n   ),\n   BoolInput(\n   name=\"should_store_message\",\n   display_name=\"Store Messages\",\n   info=\"Store the message in the history.\",\n   value=True,\n   advanced=True,\n   ),\n   DropdownInput(\n   name=\"sender\",\n   display_name=\"Sender Type\",\n   options=[MESSAGE_SENDER_AI, MESSAGE_SENDER_USER],\n   value=MESSAGE_SENDER_AI,\n   advanced=True,\n   info=\"Type of sender.\",\n   ),\n   MessageTextInput(\n   name=\"sender_name\",\n   display_name=\"Sender Name\",\n   info=\"Name of the sender.\",\n   value=MESSAGE_SENDER_NAME_AI,\n   advanced=True,\n   ),\n

MessageTextInput(\n name=\"session_id\",\n display_name=\"Session ID\",\n info=\"The session ID of the chat. If empty, the current session ID parameter will be used.\",\n advanced=True,\n),\n MessageTextInput(\n name=\"context_id\",\n display_name=\"Context ID\",\n info=\"The context ID of the chat. Adds an extra layer to the local memory.\",\n value=\"\",\n advanced=True,\n),\n MessageTextInput(\n name=\"data_template\",\n display_name=\"Data Template\",\n value=\"{text}\",\n advanced=True,\n info=\"Template to convert Data to Text. If left empty, it will be dynamically set to the Data's text key.\",\n),\n BoolInput(\n name=\"clean_data\",\n display_name=\"Basic Clean Data\",\n value=True,\n advanced=True,\n info=\"Whether to clean data before converting to string.\",\n),\n]\n outputs = [\n Output(\n display_name=\"Output Message\",\n name=\"message\",\n method=\"message_response\",\n),\n]\n\n def _build_source(self, id_: str | None, display_name: str | None, source: str | None) -> Source:\n source_dict = {}\n if id_:\n source_dict[\"id\"] = id_\n if display_name:\n source_dict[\"display_name\"] = display_name\n if source:\n # Handle case where source is a ChatOpenAI object\n if hasattr(source, \"model_name\"):\n source_dict[\"source\"] = source.model_name\n elif hasattr(source, \"model\"):\n source_dict[\"source\"] = str(source.model)\n else:\n source_dict[\"source\"] = str(source)\n return Source(**source_dict)\n\n async def message_response(self) -> Message:\n # First convert the input to string if needed\n text = self.convert_to_string()\n\n # Get source properties\n source, _, display_name, source_id = self.get_properties_from_source_component()\n\n # Create or use existing Message object\n if isinstance(self.input_value, Message) and not self.is_connected_to_chat_input():\n message = self.input_value\n # Update message properties\n message.text = text\n # Preserve existing session_id from the incoming message if it exists\n existing_session_id = message.session_id\n else:\n message = Message(text=text)\n existing_session_id = None\n\n # Set message properties\n message.sender = self.sender\n message.sender_name = self.sender_name\n # Preserve session_id from incoming message, or use component/graph session_id\n message.session_id = (\n self.session_id or existing_session_id or (self.graph.session_id if hasattr(self, \"graph\") else None) or \"\"\n)\n message.context_id = self.context_id\n message.flow_id = self.graph.flow_id if hasattr(self, \"graph\") else None\n message.properties.source = self._build_source(source_id, display_name, source)\n\n # Store message if needed\n if message.session_id and self.should_store_message:\n stored_message = await self.send_message(message)\n self.message.value = stored_message\n message = stored_message\n\n # Set accumulated token usage from all upstream LLM vertices.\n # This must happen AFTER send_message() because streaming captures\n # usage from chunks and would overwrite accumulated totals.\n if hasattr(self, \"_vertex\") and self._vertex is not None:\n accumulated_usage = self._vertex._accumulate_upstream_token_usage() # noqa: SLF001\n if accumulated_usage:\n message.properties.usage = accumulated_usage\n if self.should_store_message and message.get_id():\n message = await self._update_stored_message(message)\n await self._send_message_event(message, id_=message.get_id())\n\n self.status = message\n return message\n\n def _serialize_data(self, data: Data) -> str:\n \"\"\"Serialize Data object to JSON string.\"\"\"\n # Convert data.data to JSON-serializable format\n serializable_data = jsonable_encoder(data.data)\n # Serialize with orjson, enabling pretty printing with indentation\n json_bytes = orjson.dumps(serializable_data, option=orjson.OPT_INDENT_2)\n # Convert bytes to string and wrap in Markdown code blocks\n return \"```json\\n\" + json_bytes.decode(\"utf-8\") + \"\\n```\"\n\n def _validate_input(self) -> None:\n

\"\"\"Validate the input data and raise ValueError if invalid.\"\"\"\n if self.input_value is None:\n msg = \"Input data cannot be None\"\n raise ValueError(msg)\n if isinstance(self.input_value, list) and not all(\n isinstance(item, Message | Data | DataFrame | str) for item in self.input_value\n):\n invalid_types = [\n type(item).__name__\n for item in self.input_value\n if not isinstance(item, Message | Data | DataFrame | str)\n]\n msg = f\"Expected Data or DataFrame or Message or str, got {invalid_types}\"\n raise TypeError(msg)\n if not isinstance(\n self.input_value,\n Message | Data | DataFrame | str | list | Generator | type(None),\n):\n type_name = type(self.input_value).__name__\n msg = f\"Expected Data or DataFrame or Message or str, Generator or None, got {type_name}\"\n raise TypeError(msg)\n\n def convert_to_string(self) -> str | Generator[Any, None, None]:\n \"\"\"Convert input data to string with proper error handling.\"\"\"\n self._validate_input()\n if isinstance(self.input_value, list):\n clean_data: bool = getattr(self, \"clean_data\", False)\n return \"\\n\".join([safe_convert(item, clean_data=clean_data) for item in self.input_value])\n if isinstance(self.input_value, Generator):\n return self.input_value\n return safe_convert(self.input_value)\n"

},

"context_id": {

"_input_type": "MessageTextInput",

"advanced": true,

"display_name": "Context ID",

"dynamic": false,

"info": "The context ID of the chat. Adds an extra layer to the local memory.",

"input_types": [

"Message"

],

"list": false,

"list_add_label": "Add More",

"load_from_db": false,

"name": "context_id",

"override_skip": false,

"placeholder": "",

"required": false,

"show": true,

"title_case": false,

"tool_mode": false,

"trace_as_input": true,
"trace_as_metadata": true,
"track_in_telemetry": false,
"type": "str",
"value": ""
},
"data_template": {
"_input_type": "MessageTextInput",
"advanced": true,
"display_name": "Data Template",
"dynamic": false,
"info": "Template to convert Data to Text. If left empty, it will be dynamically set to the Data's text key.",
"input_types": [
"Message"
],
"list": false,
"list_add_label": "Add More",
"load_from_db": false,
"name": "data_template",
"override_skip": false,
"placeholder": "",
"required": false,
"show": true,
"title_case": false,
"tool_mode": false,
"trace_as_input": true,
"trace_as_metadata": true,
"track_in_telemetry": false,
"type": "str",
"value": "{text}"

},
"input_value": {
"_input_type": "HandleInput",
"advanced": false,
"display_name": "Inputs",
"dynamic": false,
"info": "Message to be passed as output.",
"input_types": [
"Data",
"JSON",
"DataFrame",
"Table",
"Message"
],
"list": false,
"list_add_label": "Add More",
"name": "input_value",
"override_skip": false,
"placeholder": "",
"required": true,
"show": true,
"title_case": false,
"trace_as_metadata": true,
"track_in_telemetry": false,
"type": "other",
"value": ""
},
"sender": {
"_input_type": "DropdownInput",
"advanced": true,
"combobox": false,

"dialog_inputs": {},
"display_name": "Sender Type",
"dynamic": false,
"external_options": {},
"info": "Type of sender.",
"name": "sender",
"options": [
"Machine",
"User"
],
"options_metadata": [],
"override_skip": false,
"placeholder": "",
"required": false,
"show": true,
"title_case": false,
"toggle": false,
"tool_mode": false,
"trace_as_metadata": true,
"track_in_telemetry": true,
"type": "str",
"value": "Machine"
},
"sender_name": {
"_input_type": "MessageTextInput",
"advanced": true,
"display_name": "Sender Name",
"dynamic": false,
"info": "Name of the sender.",
"input_types": [
"Message"

```
    ],
    "list": false,
    "list_add_label": "Add More",
    "load_from_db": false,
    "name": "sender_name",
    "override_skip": false,
    "placeholder": "",
    "required": false,
    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_input": true,
    "trace_as_metadata": true,
    "track_in_telemetry": false,
    "type": "str",
    "value": "AI"
  },
  "session_id": {
    "_input_type": "MessageTextInput",
    "advanced": true,
    "display_name": "Session ID",
    "dynamic": false,
    "info": "The session ID of the chat. If empty, the current session ID parameter will be used.",
    "input_types": [
      "Message"
    ],
    "list": false,
    "list_add_label": "Add More",
    "load_from_db": false,
    "name": "session_id",
    "override_skip": false,
```

"placeholder": "",
"required": false,
"show": true,
"title_case": false,
"tool_mode": false,
"trace_as_input": true,
"trace_as_metadata": true,
"track_in_telemetry": false,
"type": "str",
"value": ""
},
"should_store_message": {
"_input_type": "BoolInput",
"advanced": true,
"display_name": "Store Messages",
"dynamic": false,
"info": "Store the message in the history.",
"list": false,
"list_add_label": "Add More",
"name": "should_store_message",
"override_skip": false,
"placeholder": "",
"required": false,
"show": true,
"title_case": false,
"tool_mode": false,
"trace_as_metadata": true,
"track_in_telemetry": true,
"type": "bool",
"value": true
}

```
          },
          "tool_mode": false
        },
        "showNode": false,
        "type": "ChatOutput"
      },
      "dragging": false,
      "id": "ChatOutput-JQB33",
      "measured": {
        "height": 51,
        "width": 192
      },
      "position": {
        "x": 1089,
        "y": 273
      },
      "selected": false,
      "type": "genericNode"
    }
  ],
  "viewport": {
    "x": 23.403846153846132,
    "y": 77.48397435897436,
    "zoom": 0.7756410256410257
  }
},
"description": "Your Passport to Linguistic Landscapes.",
"endpoint_name": null,
"id": "1f8e0333-3ed4-44d2-8980-d203af7ebcc5",
"is_component": false,
"last_tested_version": "1.9.0",
```

"locked": false,
"name": "Testcase review",
"tags": []
}